REACT.JS SENIOR & STAFF INTERVIEW ARCHITECTURE GUIDE

Curated High-Impact Conceptual, Fiber, Concurrency & Performance Questions
Derived from sudheerj/reactjs-interview-questions

#1 [Q0] What is Concurrent Rendering? (Legacy)

Category: React 18 / 19 & Server Components | Level: Senior / Staff

The Concurrent rendering makes React apps to be more responsive by rendering component trees without blocking the main UI thread. It allows React to interrupt a long-running render to handle a high-priority event. i.e, When you enabled concurrent Mode, React will keep an eye on other tasks that need to be done, and if there's something with a higher priority it will pause what it is currently rendering and let the other task finish first. You can enable this in two ways,

```
// 2. Whole app using createRoot
  ReactDOM.unstablecreateRoot(domNode).render(<App />);
  ```
```

## #2 [Q19] What is React Fiber?

*Category: Virtual DOM, Fiber & Reconciliation | Level: Senior / Staff*

React Fiber is the new reconciliation engine in React, introduced in React 16. It's a complete rewrite of React's core algorithm(old stack-based algorithm) for rendering and updating the UI. Fiber enhances React's ability to handle asynchronous rendering, prioritized updates(assign priority to different types of updates), and interruption(ability to pause, abort, or reuse work) of rendering work, enabling smoother and more responsive user interfaces.

## #3 [Q20] What is the main goal of React Fiber?

*Category: Virtual DOM, Fiber & Reconciliation | Level: Senior / Staff*

The goal of React Fiber is to increase its suitability for areas like animation, layout, and gestures. Its headline feature is incremental rendering: the ability to split rendering work into chunks and spread it out over multiple frames.

Its main goals are:

* Incremental Rendering – Breaks work into chunks for smoother updates.
  * Interruptible Rendering – Pauses and resumes rendering to keep the UI responsive.
  * Prioritization – Handles high-priority updates (e.g. animations) before low-priority ones.
  * Concurrency Support – Enables working on multiple UI versions simultaneously.
  * Better Error Handling – Supports component-level error boundaries.
  * Suspense Support – Allows waiting for async data before rendering.
  * Improved DevTools – Enables better debugging and performance tracking.

## #4 [Q28] What is reconciliation?

*Category: Virtual DOM, Fiber & Reconciliation | Level: Senior / Staff*

`Reconciliation` is the process through which React updates the Browser DOM and makes React work faster. React use a `diffing algorithm` so that component updates are predictable and faster. React would first calculate the difference between the `real DOM` and the copy of DOM `(Virtual DOM)` when there's an update of components.

React stores a copy of Browser DOM which is called `Virtual DOM`. When we make changes or add data, React creates a new Virtual DOM and compares it with the previous one. This comparison is done by `Diffing Algorithm`.

Now React compares the Virtual DOM with Real DOM. It finds out the changed nodes and updates only the changed nodes in Real DOM leaving the rest nodes as it is. This process is called Reconciliation.

## #5 [Q132] How to add multiple middlewares to Redux?

*Category: Redux & State Management | Level: Senior / Staff*

You can use `applyMiddleware()`.

For example, you can add `redux-thunk` and `logger` passing them as arguments to `applyMiddleware()`:

## #6 [Q224] What are typical middleware choices for handling asynchronous calls in Redux?

*Category: Redux & State Management | Level: Senior / Staff*
```

Some of the popular middleware choices for handling asynchronous calls in Redux eco system are `Redux Thunk, Redux Promise, Redux Saga`.

#7 [Q230] Explain concurrent rendering with an example

Category: React 18 / 19 & Server Components | Level: Senior / Staff

Concurrent rendering in React 18 lets React prepare UI updates in a non-blocking way. This means React can pause low-priority rendering work, handle urgent updates first (like typing or clicking), and then continue rendering.

Example: Search input stays responsive while filtering a large list

const items = Array.from({ length: 10000 }, (, i) => `Item ${i + 1}`);

#8 [Q231] What is the difference between async mode and concurrent mode?

Category: React 18 / 19 & Server Components | Level: Senior / Staff

Both refers the same thing. Previously concurrent Mode being referred to as "Async Mode" by React team. The name has been changed to highlight React's ability to perform work on different priority levels. So it avoids the confusion from other approaches to Async Rendering.

#9 [Q239] What are React Server components?

Category: React 18 / 19 & Server Components | Level: Senior / Staff

React Server Component is a way to write React component that gets rendered in the server-side with the purpose of improving React app performance. These components allow us to load components from the backend.

Note: React Server Components is still under development and not recommended for production yet.

#10 [Q269] When to use client and server components?

Category: React 18 / 19 & Server Components | Level: Senior / Staff

You can efficiently build nextjs application if you are aware about which part of the application needs to use client components and which other parts needs to use server components. The common cases of both client and server components are listed below:

Client components:

1. Whenever your need to add interactivity and event listeners such as onClick(), onChange(), etc to the pages
2. If you need to use State and Lifecycle Effects like useState(), useReducer(), useEffect() etc.
3. If there is a requirement to use browser-only APIs.
4. If you need to implement custom hooks that depend on state, effects, or browser-only APIs.
5. There are React Class components in the pages.

Server components:

1. If the component logic is about data fetching.
2. If you need to access backend resources directly.
3. When you need to keep sensitive information((access tokens, API keys, etc)) on the server.
4. If you want reduce client-side JavaScript and placing large dependencies on the server.

#11 [Q278] Can you explain how useState works internally?

Category: React Hooks | Level: Senior / Staff

React's hooks, including `useState`, rely on some internal machinery that keeps track of state per component and per hook call during rendering. Here's a simplified explanation of the internal mechanics:

1. Hook List / Linked List

* React maintains a linked list or array of "hook states" for each component.
  * When a component renders, React keeps track of which hook it is currently processing via a cursor/index.
  * Each call to `useState()` corresponds to one "slot" in this list.

#12 [Q309] How does React Fiber works? Explain in detail.

Category: React 18 / 19 & Server Components | Level: Senior / Staff

React Fiber is the core engine that enables advanced features like concurrent rendering, prioritization, and interruptibility in React. Here's how it works:

1. Fiber Tree Structure

Each component in your app is represented by a Fiber node in a tree structure. A Fiber node contains:
* Component type
* Props & state
* Pointers to parent, child, and sibling nodes
* Effect tags to track changes (e.g., update, placement)
* This forms the Fiber Tree, a data structure React uses instead of the traditional call stack.

2. Two Phases of Rendering

A. Render Phase (work-in-progress)

* React builds a work-in-progress Fiber tree.
* It walks through each component (begin phase), calculates what needs to change, and collects side effects (complete phase).
* This phase is interruptible—React can pause it and resume later.

B. Commit Phase

* React applies changes to the Real DOM.
* Runs lifecycle methods (e.g., `componentDidMount`, `useEffect`).
* This phase is non-interruptible but fast.

3. Work Units and Scheduling

* React breaks rendering into units of work (small tasks).
* These units are scheduled based on priority using the React Scheduler.
* If time runs out (e.g., user starts typing), React can pause and yield control back to the browser.

4. Double Buffering with Two Trees

* React maintains two trees:
* Current Tree – what's visible on the screen.
* Work-In-Progress Tree – the next version being built in memory.
* Only after the new tree is fully ready, React commits it, making it the new current tree.

5. Concurrency and Prioritization

* React can prepare multiple versions of UI at once (e.g., during slow data loading).
* Updates can be assigned priorities, so urgent updates (like clicks) are handled faster than background work.

#13 [Q315] How do you share state logic between components using custom hooks?

Category: React Hooks | Level: Senior / Staff

Custom hooks allow you to extract and share stateful logic between components without changing their hierarchy. The state itself is not shared—each component using the hook gets its own isolated state.

Example: useLocalStorage Hook

```jsx
import { useState, useEffect } from 'react';

function useLocalStorage(key, initialValue) {
  // Get stored value or use initial value
  const [storedValue, setStoredValue] = useState(() => {
    try {
      const item = window.localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch (error) {
      console.error(error);
```

```
      return initialValue;
    }
  });
```

#14 [Q338] What is Redux middleware?

Category: Redux & State Management | Level: Senior / Staff

Middleware provides a way to extend Redux with custom logic that runs between dispatching an action and the moment it reaches the reducer. It's the standard extension point for side effects (logging, crash reporting, async calls, analytics) since reducers themselves must stay pure and synchronous.

```
const logger = (store) => (next) => (action) => {
  console.log("dispatching", action);
  const result = next(action);
  console.log("next state", store.getState());
  return result;
};
const store = configureStore({
  reducer: rootReducer,
  middleware: (getDefaultMiddleware) => getDefaultMiddleware().concat(logger),
});
```

#15 [Q339] What is the Redux middleware signature?

Category: Redux & State Management | Level: Senior / Staff

Middleware is written as three nested (curried) functions:

- `store` gives access to `getState()` and `dispatch()`.
  - `next` passes the action along to the next middleware (or the root reducer, if this is the last middleware).
  - `action` is the action object being processed.

Middleware must call `next(action)` (or `dispatch(...)` for a different action) to keep the chain moving — forgetting to call `next` silently swallows every action.

#16 [Q13] What is context?

Category: Virtual DOM, Fiber & Reconciliation | Level: Mid-Level

Context provides a way to pass data through the component tree without having to pass props down manually at every level.

For example, authenticated users, locale preferences, UI themes need to be accessed in the application by many components.

#17 [Q27] What is the purpose of `getDerivedStateFromProps()` lifecycle method?

Category: Virtual DOM, Fiber & Reconciliation | Level: Mid-Level

The new static `getDerivedStateFromProps()` lifecycle method is invoked after a component is instantiated as well as before it is re-rendered. It can return an object to update state, or `null` to indicate that the new props do not require any state updates.

This lifecycle method along with `componentDidUpdate()` covers all the use cases of `componentWillReceiveProps()`.

#18 [Q29] Does the lazy function support named exports?

Category: Virtual DOM, Fiber & Reconciliation | Level: Mid-Level

No, currently `React.lazy` function supports default exports only. If you would like to import modules which are named exports, you can create an intermediate module that reexports it as the default. It also ensures that tree shaking keeps working and don't pull unused components.

Let's take a component file which exports multiple named components,

and reexport `MoreComponents.js` components in an intermediate file `IntermediateComponent.js`

Now you can import the module using lazy function as below,

#19 [Q51] Do Hooks replace render props and higher order components?

Category: Virtual DOM, Fiber & Reconciliation | Level: Mid-Level

Both render props and higher-order components render only a single child but in most of the cases Hooks are a simpler way to serve this by reducing nesting in your tree.

#20 [Q51] How to use `connect()` from React Redux?

Category: Redux & State Management | Level: Mid-Level

You need to follow two steps to use your store in your container:

1. Use `mapStateToProps()`: It maps the state variables from your store to the props that you specify.
2. Connect the above props to your container: The object returned by the `mapStateToProps` function is connected to the container. You can import `connect()` from `react-redux`.

```
class App extends React.Component {
      render() {
        return <div>{this.props.containerData}</div>;
      }
     }
```

#21 [Q52] Whats the purpose of `at` symbol in the Redux connect decorator?

Category: Redux & State Management | Level: Mid-Level

The @ symbol is in fact a JavaScript expression used to signify decorators. Decorators make it possible to annotate and modify classes and properties at design time.

Let's take an example setting up Redux without and with a decorator.

- Without decorator:

#22 [Q75] Give an example on How to use context?

Category: Virtual DOM, Fiber & Reconciliation | Level: Mid-Level

Context is designed to share data that can be considered global for a tree of React components.

For example, in the code below lets manually thread through a “theme” prop in order to style the Button component.

#23 [Q78] How do you solve performance corner cases while using context?

Category: Virtual DOM, Fiber & Reconciliation | Level: Mid-Level

The context uses reference identity to determine when to re-render, there are some gotchas that could trigger unintentional renders in consumers when a provider’s parent re-renders.

For example, the code below will re-render all consumers every time the Provider re-renders because a new object is always created for value.

This can be solved by lifting up the value to parent state,

#24 [Q88] How do you get redux scaffolding using create-react-app?

Category: Redux & State Management | Level: Mid-Level

Redux team has provided official redux+js or redux+typescript templates for create-react-app project. The generated project setup includes,

1. Redux Toolkit and React-Redux dependencies
2. Create and configure Redux store
3. React-Redux `<Provider>` passing the store to React components
4. Small "counter" example to demo how to add redux logic and React-Redux hooks API to interact with the store from components

The below commands need to be executed along with template option as below,

5. Javascript template:

```js
npx create-react-app my-app --template redux
```

2. Typescript template:

```js
npx create-react-app my-app --template redux-typescript
```

#25 [Q103] What is Redux?

Category: Redux & State Management | Level: Mid-Level

Redux is a predictable state container for JavaScript applications, most commonly used with React. It helps you manage and centralize your application's state in a single source of truth, enabling easier debugging, testing, and maintenance—especially in large or complex applications. Redux core is tiny library(about 2.5kB gzipped) and has no dependencies.

#26 [Q104] What are the core principles of Redux?

Category: Virtual DOM, Fiber & Reconciliation | Level: Mid-Level

Redux follows three fundamental principles:

1. Single source of truth: The state of your whole application is stored in an object tree within a single store. The single state tree makes it easier to keep track of changes over time and debug or inspect the application.

```jsx
const store = createStore(reducer);
```

2. State is read-only: The only way to change the state is to emit an action, an object describing what happened. This ensures that neither the views nor the network callbacks will ever write directly to the state.

```js
const action = { type: 'INCREMENT' };
store.dispatch(action);
```

3. Changes are made with pure functions(Reducers): To specify how the state tree is transformed by actions, you write reducers. Reducers are just pure functions that take the previous state and an action as parameters, and return the next state.

```jsx
function counter(state = 0, action) {
  switch (action.type) {
    case 'INCREMENT':
      return state + 1;
    case 'DECREMENT':
      return state - 1;
    default:
      return state;
  }
}
```

#27 [Q105] What are the downsides of Redux compared to Flux?

Category: Redux & State Management | Level: Mid-Level

While Redux offers a powerful and predictable state management solution, it comes with a few trade-offs when compared to Flux. These include:

1. Immutability is essential

   Redux enforces a strict immutability model for state updates, which differs from Flux's more relaxed approach. This means you must avoid mutating state directly. Many Redux-related libraries assume immutability, so your team must be disciplined in writing pure update logic. You can use tools like `redux-immutable-state-invariant`, `Immer`, or `Immutable.js` to help enforce this practice, especially during development.

2. Careful selection of complementary packages

   Redux is more minimal by design and provides extension points such as middleware and store enhancers. This has led to a large ecosystem, but it also means you must thoughtfully choose and configure additional packages for features like undo/redo, persistence, or form handling—something Flux explicitly leaves out but may be simpler to manage in smaller setups.

3. Limited static type integration

   While Flux has mature support for static type checking with tools like Flow, Redux's type integration is less seamless. Although TypeScript is commonly used with Redux now, early Flow support was limited, and more boilerplate was required for static type safety. This may affect teams that rely heavily on type systems for large codebases.

#28 [Q108] How to access Redux store outside a component?

Category: Redux & State Management | Level: Mid-Level

You just need to export the store from the module where it created with `createStore()`. Also, it shouldn't pollute the global window object.

export default store;
```

### #29 [Q110] Are there any similarities between Redux and RxJS?

*Category: Redux & State Management | Level: Mid-Level*

These libraries are very different for very different purposes, but there are some vague similarities.

Redux is a tool for managing state throughout the application. It is usually used as an architecture for UIs. Think of it as an alternative to (half of) Angular. RxJS is a reactive programming library. It is usually used as a tool to accomplish asynchronous tasks in JavaScript. Think of it as an alternative to Promises. Redux uses the Reactive paradigm because the Store is reactive. The Store observes actions from a distance, and changes itself. RxJS also uses the Reactive paradigm, but instead of being an architecture, it gives you basic building blocks, Observables, to accomplish this pattern.

### #30 [Q111] How to reset state in Redux?

*Category: Redux & State Management | Level: Mid-Level*

You need to write a root reducer in your application which delegate handling the action to the reducer generated by `combineReducers()`.

For example, let us take `rootReducer()` to return the initial state after `USERLOGOUT` action. As we know, reducers are supposed to return the initial state when they are called with `undefined` as the first argument, no matter the action.

```
const rootReducer = (state, action) => {
  if (action.type === "USERLOGOUT") {
    state = undefined;
  }
```

#31 [Q112] What is the difference between React context and React Redux?

Category: Redux & State Management | Level: Mid-Level

You can use Context in your application directly and is going to be great for passing down data to deeply nested components which what it was designed for.

Whereas Redux is much more powerful and provides a large number of features that the Context API doesn't provide. Also, React Redux uses context internally but it doesn't expose this fact in the public API.

Aspect	Context API	Redux
Purpose	Dependency injection — passes data through the tree without prop drilling	Full state-management library with a predictable, centralized store
State updates	Plain `useState`/`useReducer` next to the provider; no built-in middleware	Actions + reducers, with middleware support (`redux-thunk`, `redux-saga`, etc.)
Performance	Every consumer re-renders on any value change unless you split contexts/memoize	Uses selectors (`useSelector`/`connect`) so components only re-render when the selected slice changes
DevTools	None built-in	Time-travel debugging, action logs via Redux DevTools
Async logic	You wire it yourself (custom hooks, effects)	Standardized patterns via middleware
Boilerplate	Minimal — just `createContext`/`useContext`	More setup, though Redux Toolkit reduces this significantly

#32 [Q113] Why are Redux state functions called reducers?

Category: Redux & State Management | Level: Mid-Level

Reducers always return the accumulation of the state (based on all previous and current actions). Therefore, they act as a reducer of state. Each time a Redux reducer is called, the state and action are passed as parameters. This state is then reduced (or accumulated) based on the action, and then the next state is returned. You could reduce a collection of actions and an initial state (of the store) on which to perform these actions to get the resulting final state.

#33 [Q114] How to make AJAX request in Redux?
```

Category: Redux & State Management | Level: Mid-Level

You can use `redux-thunk` middleware which allows you to define async actions.

Let's take an example of fetching specific account as an AJAX call using fetch API:

```
function setAccount(data) {
    return { type: "SETAccount", data: data };
  }
  ```
```

### #34 [Q115] Should I keep all component's state in Redux store?

*Category: Redux & State Management | Level: Mid-Level*

Keep your data in the Redux store, and the UI related state internally in the component.

### #35 [Q116] What is the proper way to access Redux store?

*Category: Redux & State Management | Level: Mid-Level*

The best way to access your store in a component is to use the `connect()` function, that creates a new component that wraps around your existing one. This pattern is called Higher-Order Components, and is generally the preferred way of extending a component's functionality in React. This allows you to map state and action creators to your component, and have them passed in automatically as your store updates.

Let's take an example of `<FilterLink>` component using connect:

```
const mapStateToProps = (state, ownProps) => ({
    active: ownProps.filter === state.visibilityFilter,
  });
```

#36 [Q117] What is the difference between component and container in React Redux?

Category: Redux & State Management | Level: Mid-Level

Component is a class or function component that describes the presentational part of your application.

Container is an informal term for a component that is connected to a Redux store. Containers subscribe to Redux state updates and dispatch actions, and they usually don't render DOM elements; they delegate rendering to presentational child components.

#37 [Q118] What is the purpose of the constants in Redux?

Category: Redux & State Management | Level: Mid-Level

Constants allows you to easily find all usages of that specific functionality across the project when you use an IDE. It also prevents you from introducing silly bugs caused by typos – in which case, you will get a `ReferenceError` immediately.

Normally we will save them in a single file (`constants.js` or `actionTypes.js`).

In Redux, you use them in two places:

#38 [Q121] How to structure Redux top level directories?

Category: Redux & State Management | Level: Mid-Level

Most of the applications has several top-level directories as below:

1. Components: Used for dumb components unaware of Redux.
2. Containers: Used for smart components connected to Redux.
3. Actions: Used for all action creators, where file names correspond to part of the app.
4. Reducers: Used for all reducers, where files name correspond to state key.
5. Store: Used for store initialization.

This structure works well for small and medium size apps.

#39 [Q122] What is redux-saga?

Category: Redux & State Management | Level: Mid-Level

`redux-saga` is a library that aims to make side effects (asynchronous things like data fetching and impure things like accessing the browser cache) in React/Redux applications easier and better.

It is available in NPM:
```

#40 [Q123] What is the mental model of redux-saga?

Category: Redux & State Management | Level: Mid-Level

Saga is like a separate thread in your application, that's solely responsible for side effects. `redux-saga` is a redux middleware, which means this thread can be started, paused and cancelled from the main application with normal Redux actions, it has access to the full Redux application state and it can dispatch Redux actions as well.

#41 [Q124] What are the differences between `call()` and `put()` in redux-saga?

Category: Redux & State Management | Level: Mid-Level

Both `call()` and `put()` are effect creator functions. `call()` function is used to create effect description, which instructs middleware to call the promise. `put()` function creates an effect, which instructs middleware to dispatch an action to the store.

Let's take example of how these effects work for fetching particular user data.

```
// Instructing middleware to dispatch corresponding action.
    yield put({
      type: "FETCHUSERSUCCESS",
      userData,
    });
  }
  ```
```

### #42 [Q125] What is Redux Thunk?

*Category: Redux & State Management | Level: Mid-Level*

Redux Thunk middleware allows you to write action creators that return a function instead of an action. The thunk can be used to delay the dispatch of an action, or to dispatch only if a certain condition is met. The inner function receives the store methods `dispatch()` and `getState()` as parameters.

### #43 [Q126] What are the differences between `redux-saga` and `redux-thunk`?

*Category: Redux & State Management | Level: Mid-Level*

Both Redux Thunk and Redux Saga take care of dealing with side effects. In most of the scenarios, Thunk uses Promises to deal with them, whereas Saga uses Generators. Thunk is simple to use and Promises are familiar to many developers, Sagas/Generators are more powerful but you will need to learn them. But both middleware can coexist, so you can start with Thunks and introduce Sagas when/if you need them.

### #44 [Q127] What is Redux DevTools?

*Category: Redux & State Management | Level: Mid-Level*

Redux DevTools is a live-editing time travel environment for Redux with hot reloading, action replay, and customizable UI. If you don't want to bother with installing Redux DevTools and integrating it into your project, consider using Redux DevTools Extension for Chrome and Firefox.

### #45 [Q128] What are the features of Redux DevTools?

*Category: Redux & State Management | Level: Mid-Level*

Some of the main features of Redux DevTools are below,

1. Lets you inspect every state and action payload.
2. Lets you go back in time by cancelling actions.
3. If you change the reducer code, each staged action will be re-evaluated.
4. If the reducers throw, you will see during which action this happened, and what the error was.
5. With `persistState()` store enhancer, you can persist debug sessions across page reloads.

### #46 [Q129] What are Redux selectors and why use them?

*Category: Redux & State Management | Level: Mid-Level*

Selectors are functions that take Redux state as an argument and return some data to pass to the component.

For example, to get user details from the state:

These selectors have two main benefits,
```

#47 [Q130] What is Redux Form?

Category: Redux & State Management | Level: Mid-Level

Redux Form works with React and Redux to enable a form in React to use Redux to store all of its state. Redux Form can be used with raw HTML5 inputs, but it also works very well with common UI frameworks like Material UI, React Widgets and React Bootstrap.

#48 [Q131] What are the main features of Redux Form?

Category: Redux & State Management | Level: Mid-Level

Some of the main features of Redux Form are:

1. Field values persistence via Redux store.
2. Validation (sync/async) and submission.
3. Formatting, parsing and normalization of field values.

#49 [Q133] How to set initial state in Redux?

Category: Redux & State Management | Level: Mid-Level

You need to pass initial state as second argument to createStore:

```
const initialState = {
  todos: [{ id: 123, name: "example", completed: false }],
};
const store = createStore(rootReducer, initialState);
```

#50 [Q134] How Relay is different from Redux?

Category: Redux & State Management | Level: Mid-Level

Relay is similar to Redux in that they both use a single store. The main difference is that relay only manages state originated from the server, and all access to the state is used via GraphQL queries (for reading data) and mutations (for changing data). Relay caches the data for you and optimizes data fetching for you, by fetching only changed data and nothing more.